

Security Remediation Report

Project-Ticket (Fute Portal) — for Cybersecurity / Application Security review

Prepared	2026-08-24
Source audit	docs/PRODUCTION_READINESS_AUDIT.md — 13-agent multi-pass audit, 24 findings verified against source
Scope	main/backend (Express + Firestore), main/frontend (React SPA)
Verification method	Static code verification only — route-by-route HTTP status checks, export/import matching, esbuild transform checks. No live browser click-through was performed this session (see Verification Limitations).

1. Purpose

This report documents every fix applied in response to the production-readiness audit, organized by security relevance, so the Cybersecurity team can independently assess what changed, why it closes a real risk, and what still needs attention. It is written for review, not as a claim of a completed penetration test.

2. Security & Session Fixes (Primary Interest)

2.1 Session revocation did not actually work

Item	Detail
Finding IDs	F04 (HIGH), F06 (MEDIUM)
Problem	The frontend axios client had no response interceptor to react to a 401. An admin using the Security Center's "Force logout" / "Revoke session" action revoked the session server-side, but the victim's already-open browser tab kept working — the SPA never re-checked the token, so a revoked or expired (7-day) JWT stayed functionally "logged in" client-side. Separately, clicking Logout only cleared browser storage; there was no server-side call to invalidate the session record, so a copied token (devtools, shared machine, synced backup) remained valid for up to 7 days after the user logged out.
Fix	<code>main/frontend/src/utils/api.js</code> — added a response interceptor: any 401 (except on the login/register endpoints themselves) clears local storage and redirects to <code>/login</code>. <code>main/backend/controllers/authController.js</code> — new <code>logout()</code> handler revokes the caller's own session record (<code>SESSIONS.doc(sid).set({revoked:true})</code>) using the existing session-revocation machinery the Security Center already relied on. <code>main/backend/middleware/authMiddleware.js</code> — the JWT's session id (<code>sid</code>) is now attached to <code>req.user</code> so the logout handler can identify which session to revoke. <code>main/frontend/src/context/AuthContext.jsx</code> — <code>logout()</code> now calls the new <code>POST /api/auth/logout</code> endpoint (best-effort; local state still clears even if the network call fails).
Result	An admin's "force logout" now takes effect on the next request from that session; a user's own Logout revokes the token server-side instead of only forgetting it locally.

2.2 Credentials file sitting in the repo working tree

Item	Detail
Finding IDs	F05 (HIGH), F09 (LOW)
Problem	<code>docs/login-credentials.pdf</code> — a file literally named for credentials — was untracked and not covered by any <code>.gitignore</code> rule, one <code>git add</code> away from being permanently committed to git history.

Fix	Added <code>docs/*credential* / docs/*Credential*</code> to <code>.gitignore</code> . Confirmed via <code>git check-ignore -v</code> that the file is now excluded from staging.
Residual action for this team	The file itself was not deleted (not this assistant's data to remove without explicit instruction). If it was ever committed on any branch in the past, treat every credential inside it as compromised and rotate — a <code>.gitignore</code> rule only prevents future commits, it does not scrub history.

2.3 Unthrottled, unbounded expensive reads

Item	Detail
Finding IDs	F03 (HIGH), F15 (MEDIUM), F17 (MEDIUM)
Problem	The default (no date-range filter) Analytics load, the Super Admin dashboard overview, and the SLA compliance view each performed full, uncapped Firestore collection scans with no rate limit and no caching — reachable repeatedly by any founder/superadmin session.
Fix	- Every scan in <code>analyticsController.js</code>, <code>dashboardController.js</code>, and <code>slaController.js</code> now carries an explicit <code>.limit()</code> cap. 60-second (analytics) / 30-second (dashboard) in-process TTL caches added, matching the pattern already used by <code>authMiddleware</code>'s profile cache. - A shared <code>express-rate-limit</code> (20 req/min) was added to <code>/analytics</code>, <code>/analytics/export</code>, <code>/dashboard-overview</code>, and <code>/sla-compliance</code> in <code>founderRoutes.js</code> as a backstop against a caller that varies query params to bypass the cache.
Result	Read cost and latency for these endpoints are now bounded and rate-limited instead of scaling linearly, unbounded, with total historical row count.

2.4 Missing rate limit on a secondary auth endpoint

Item	Detail
Finding ID	F08 (LOW)
Problem	<code>POST /api/auth/verify-password</code> (used for risk-tiered re-authentication confirm dialogs) had no rate limit, unlike <code>/login</code> . An attacker holding a stolen JWT could brute-force the account's real password against this endpoint with no throttle.
Fix	The existing <code>authLimiter</code> (10 requests / 15 min per IP) is now applied to this route in <code>main/backend/routes/authRoutes.js</code> .

2.5 Authorization gap — org-wide task read

Item	Detail
Finding ID	F07 (LOW)
Problem	<code>GET /api/coordinator/tasks</code> had no server-side scoping — any authenticated employee account could read the organization's entire task backlog (including PR/Figma links), not just their own assigned tasks. The frontend's "My Tasks" view was filtering client-side after already receiving everything.
Fix	<code>main/backend/controllers/taskProjectController.js</code> 's <code>getTasks</code> now adds a server-side <code>.where('assignee', '==', req.user.full_name)</code> filter when the caller's role is <code>employee</code> ; Coordinator/Founder retain the full team-wide read they legitimately need.

2.6 Weak password floor on admin-issued resets

Item	Detail
------	--------

Finding ID	F10 (LOW)
Problem	Admin-issued password resets (the password set right after an account logout) only enforced a 6-character minimum — the same weak floor as open self-registration, undermining the logout protection it's meant to restore.
Fix	Minimum raised to 10 characters, enforced both server-side (<code>superAdminUserController.js</code>) and reflected in the frontend form (<code>SuperAdminUsersPage.jsx</code>).

2.7 Silent misconfiguration on missing secrets

Item	Detail
Finding ID	F23 (MEDIUM)
Problem	<code>JWT_SECRET</code> was read lazily wherever used, with no startup check. A deployment missing this variable would start "successfully" and only fail on the first login attempt — turning a configuration mistake into a production incident instead of a failed deploy.
Fix	<code>main/backend/server.js</code> now checks <code>process.env.JWT_SECRET</code> at boot and exits immediately with a clear error if unset, before the server starts accepting traffic.

3. Reliability / Operational Hardening (Secondary Interest to Security)

Item	What changed
Health check (F24)	New <code>GET /healthz</code> performs a real Firestore read and reports reachability/latency, distinct from the existing static <code>GET /</code> . Orchestrators can now tell "process up" apart from "database reachable."
Graceful shutdown (F24)	<code>SIGTERM</code> / <code>SIGINT</code> now drain in-flight requests via <code>server.close()</code> before exiting, instead of hard-killing connections on restart/redeploy.
Silent data loss on ticket edits (F20)	A non-editable ticket field edit used to apply optimistically in the UI and silently revert with no warning on the next refresh. Now checked and rejected with a toast <i>before</i> the local UI update, closing a data-integrity gap.
Swallowed errors (F21)	<code>LeaveContext.applyLeave</code> now rethrows on failure (matching every other write path in the app), so a future caller can't show a false success state.
Truncated views with no signal (F16)	Founder-side merged HR+IT views that hit their read cap now set an <code>X-Results-Truncated</code> response header instead of silently omitting records with no indication.

4. Architecture Changes (No Behavioral Change Intended)

Two large refactors were performed for maintainability — flagged here specifically because refactors are where an unintended behavior change is most likely to slip in, which is exactly what Section 6 below addresses.

Change	Detail
Backend controller split (F01)	The 951-line, 31-handler <code>founderController.js</code> was split into 8 files by concern: <code>superAdminUserController.js</code> , <code>analyticsController.js</code> , <code>slaController.js</code> , <code>notificationController.js</code> , <code>departmentController.js</code> , <code>permissionController.js</code> , <code>systemSettingsController.js</code> , <code>dashboardController.js</code> . All 31 original exports are preserved exactly; every route was individually re-verified post-split (Section 6).
HR/IT controller de-duplication (F02)	The two nearly-identical complaint controllers were consolidated into one shared <code>complaintControllerFactory.js</code> , instantiated per-queue in <code>hrController.js</code> / <code>itController.js</code> — a bug fix now only needs to land once.
Frontend component split (F11)	The 1317-line <code>DashboardPage.jsx</code> was decomposed into <code>DataRequestsView.jsx</code> , <code>AssetsView.jsx</code> , <code>ReportsView.jsx</code> , <code>RenderingStatusView.jsx</code> , with <code>DashboardPage.jsx</code> left as a thin composition root.

Shared pagination hook (F12)	A duplicated cursor-pagination state pattern across three React contexts was extracted into <code>hooks/useCursorPagination.js</code> .
Named constants (F13)	A bare <code>.limit(200)</code> literal repeated ~18 times across 9 files was replaced with named constants in <code>utils/constants.js</code> .

5. Deliberately Deferred (Not Fixed — Tracked, Not Silent)

Finding	Why deferred
F14 — Nav permission registry hand-synched across layout files	Consolidating this touches every role's navigation layout simultaneously; without live browser verification available this session, the risk of silently breaking navigation for one role outweighed the maintainability benefit. Left with its existing tracking comment in <code>PermissionsContext.jsx</code> .
F18 — Contexts eager-fetch on login regardless of visited page	Partially mitigated already — every context role-gates and no-ops for irrelevant roles. Full fix requires moving fetch triggers from provider-mount to per-page-mount across 6 contexts and every consuming page — documented as a tracked tradeoff in <code>App.jsx</code> rather than rushed.
F22 — Login form disables native validation	The audit itself rated this "not blocking."

6. Verification Method & Limitations

What was verified:

- Every backend route (31 founder/superadmin routes plus new auth/coordinator routes) was hit with curl after every change; a dropped handler or broken `require()` would surface as a 500 or crash — confirmed 401 (auth-gated, correctly wired) across the board, not 404/500.
- Every new/changed backend file's exports were diffed against the original monolith's export list — exact match, nothing dropped.
- Every new/changed frontend file was fetched through Vite's dev server transform endpoint — a syntax error would return HTTP 500; all returned 200.
- The backend process was confirmed alive and serving (`/healthz` returning 200) after every change.

What was *not* verified — and should be, before this ships:

- No live browser session was reachable from this working environment (Chrome could reach the public internet but not the local dev servers). **No actual login → logout → 401 replay, ticket creation, or dashboard click-through was performed.**
- The two largest refactors (Section 4) are exactly the kind of change most likely to hide a wrong-prop or dropped-handler bug behind a passing compile check. Static verification (above) catches "does it run" — it does not catch "does it render/behave correctly."

Recommended before production sign-off

1. Log in, confirm normal flow, log out, confirm the old token now returns 401 instead of continuing to work.
2. Create one HR ticket and one IT ticket end-to-end.
3. Load Super Admin → Analytics / Dashboard Overview / SLA Compliance once each and sanity-check the numbers.
4. Click "Load More" on Tickets, Approvals, and Tasks once each.
5. Click through every IT dashboard tab (Dashboard, Tickets, Approval Center, Data Requests, Assets, Reports, Rendering Status).
6. Confirm `docs/login-credentials.pdf` was never previously committed to any branch; if it was, rotate every credential it contains.

7. File Inventory

Category	Files
New backend files	controllers/{superAdminUserController,analyticsController,slaController,notificationController,departmentController,permissionController,systemSettingsController,dashboardController,complaintControllerFactory}.js , utils/constants.js
Removed	controllers/founderController.js (split into the 8 files above)
Modified backend files	authController.js , hrController.js , itController.js , middleware/authMiddleware.js , routes/{authRoutes,founderRoutes}.js , server.js , plus 6 controllers touched only for the UNPAGINATED_READ_LIMIT constant swap (assetController, hrDeskController, leaveController, renderController, securityController, taskProjectController)
New frontend files	components/{DataRequestsView,AssetsView,ReportsView, RenderingStatusView}.jsx , hooks/useCursorPagination.js
Modified frontend files	utils/api.js , context/{AuthContext,ApprovalContext, LeaveContext,TaskProjectContext,TicketContext}.jsx , pages/{DashboardPage,SuperAdminUsersPage}.jsx , components/DataTable.tsx , App.jsx
Config	.gitignore

Generated from docs/PRODUCTION_READINESS_AUDIT.md and the working session that implemented its findings. This document describes the change set as of 2026-08-24; it is not a substitute for an independent security review.